Who sacked my cluster?

Sachin Singh

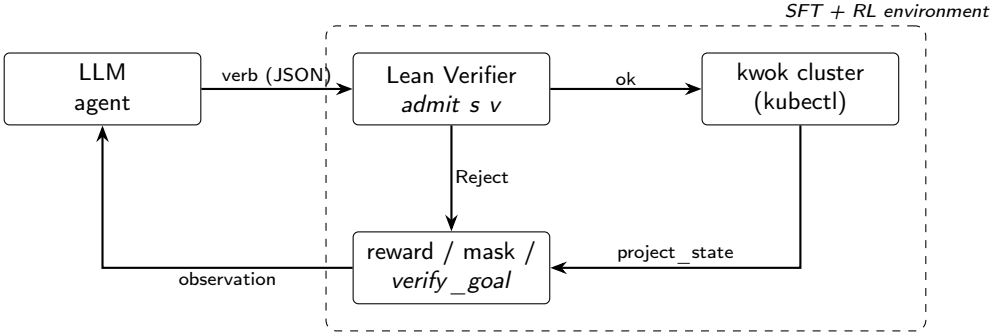
https://github.com/sachinkumarsingh092/lean-rl/tree/master/lean_rl/lean

The problem

- LLM-driven SRE agents propose operations on a Kubernetes cluster.
- The agent is non-deterministic; we cannot trust it to be safe.
- **Idea:** between "agent emits action" and "cluster executes action", there is a *Lean-checked admission gate* that accept a machine-checkable proof and gives a structured reason output that becomes RL feedback.

I've made a SFT + RL gym as a use-case for this which I used to train a small model. But I've only explained the Lean side here.

Architecture



Lean = judge | kwok = world (source of truth)

A small Kubernetes glossary

Kubernetes Open-source orchestration system for containerized workloads. Provides a declarative API for cluster objects and controllers that drive actual state toward desired state.

Pod One or more containers running together as a unit. Modeled as a `Pod` struct with `phase`, `node`, and a resource `request`.

Node A worker machine that hosts pods. Has a `capacity` (CPU and memory) and a `cordoned` flag.

Deployment Desired-state spec saying "I want N replicas of this image". The control loop tries to keep that count.

Replica One running pod belonging to a deployment. `availableReplicas` counts the Running ones.

Capacity The sum of all pod requests on a node must not exceed the node's CPU and memory capacity.

kwok A project that simulates the Kubernetes API, so we can act on a cluster without running real workloads (kwok.sigs.k8s.io).

PDB Pod Disruption Budget. A per-deployment promise that at least `minAvailable` replicas stay Running across voluntary disruptions.

Anti-affinity Scheduling rule: no two pods of the same deployment land on the same node.

resourceVersion Opaque counter that bumps on every change. The agent echoes the version it last saw, so concurrent edits cannot silently overwrite.

Admission webhook Real Kubernetes extension point where the API server calls out before persisting an object. Our `admit` is a Lean-checked admission webhook.

Drain / cordon Mark a node unschedulable, then evict its pods.

kubectl The Kubernetes command-line client. We use it to push admitted verbs to kwok.

The Lean stack

```
import ProvedSRE.Cluster      -- ADTs: Pod, Node, Deployment, State; step
import ProvedSRE.Invariants   -- pdbRespected, capacityRespected, antiAffinityRespected, safe
import ProvedSRE.Api          -- ApiVerb, apply, preCheck, postCheck, admit
import ProvedSRE.Goals        -- episode termination predicates
import ProvedSRE.Soundness    -- theorem admitSafe : admit s v = .ok () -> safe (apply s v)
import ProvedSRE.Verify       -- FromJson/ToJson + stdin/stdout JSON line server
```

Toolchain: `leanprover/lean4:v4.29.0`, *no* mathlib (pure core Lean 4).

Decidable invariants

```
def pdbRespected (s : State) : Bool :=
  s.deployments.all fun d => availableReplicas s d.id ≥ d.minAvailable

inductive InvariantKind | pdb | capacity | antiAffinity

def invariants : List (InvariantKind × (State → Bool)) :=
  [ (.pdb,          pdbRespected),
    (.capacity,     capacityRespected),
    (.antiAffinity, antiAffinityRespected) ]

def safe (s : State) : Bool := invariants.all ·(.2 s)
```

Each invariant is a `Bool` predicate on `State`. They are registered once in a small table. Adding a new invariant is a single line in that table, and `safe`, `postCheck`, and the soundness proof all pick it up automatically.

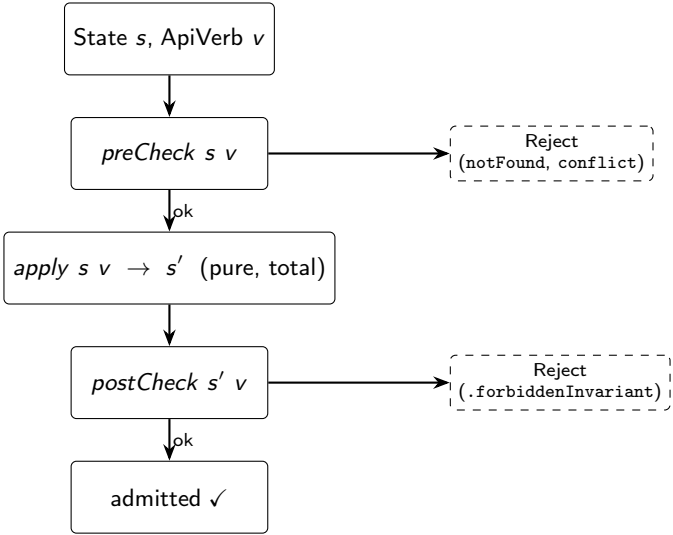
The admission gate

```
def postCheck (s′ : State) (v : ApiVerb) : Except Reject Unit :=
  invariants.foldlM (init := ()) fun _ ⟨ik, p⟩ =>
    if p s′ then .ok ()
    else .error (.forbiddenInvariant ik (verbTarget v) (invariantReason ik))

def admit (s : State) (v : ApiVerb) : Except Reject Unit :=
  match preCheck s v with
  | .error r => .error r
  | .ok _    => postCheck (apply s v) v
```

`admit` runs the pre-checks first, then dry-runs the verb with `apply` and re-evaluates every invariant in the registry. This mirrors how a Kubernetes admission webhook would dry-run an incoming request.

Inside `admit`



Everything past this gate is covered by the theorem on the next slide.

Soundness, completeness, and trace-level safety

```
theorem postCheck_iff :
    postCheck s′ v = .ok () ↔ ∀ p ∈ invariants, p.2 s′ = true

theorem admitSafe :
    admit s v = .ok () → safe (apply s v) = true

theorem admit_ok_of_safe :
    preCheck s v = .ok () → safe (apply s v) = true →
    admit s v = .ok ()

theorem runApiPlan_safe :
    safe s = true → runApiPlan s vs = .ok sf → safe sf = true
```

Soundness reduces to a single biconditional that says `postCheck` accepts a state exactly when every registered invariant holds. The completeness theorem says the gate accepts every plan that is in fact safe. The trace-level theorem lifts safety from a single verb to a full admitted plan by induction.

Episode goals

```
inductive Goal
  | drainNode    (n : NodeId)
  | rolloutImage (d : DeploymentId) (img : Image)
  | scaleTo      (d : DeploymentId) (replicas : Nat)

def goalAchieved (s : State) : Goal → Bool
  | .drainNode n        =>
      (s.pods.filter fun p => p.node = n ∧ p.phase ≠ .Failed).isEmpty
  | .rolloutImage d img =>
      (s.pods.filter fun p => p.deployment = d ∧ p.phase = .Running).all
        (fun p => p.image = img)
  | .scaleTo d n        => availableReplicas s d = n
```

Goals are typed, decidable predicates on the projected state. After every admitted verb the orchestrator evaluates `goalAchieved` to decide whether the episode is finished and the goal-completion reward is due.

The wire layer

```
def handleLine (line : String) : Json :=
  match Json.parse line with
  | .error e => errorResult "" s!"parse: {e}"
  | .ok j    =>
    match (j.getObjValAs? String "mode").toOption.getD "plan" with
    | "verb" => toJson (runVerbRequest (fromJson? j))
    | "goal" => toJson (runGoalRequest (fromJson? j))
    | _      => toJson (runRequest     (fromJson? j))  -- "plan"
```

The Lean kernel is a single binary that speaks a JSON line protocol on stdin and stdout. Each request dispatches on a `mode` field: `"verb"` admits and applies one `ApiVerb`, `"goal"` evaluates `goalAchieved`, and `"plan"` walks a full trajectory and reports the first invariant violation. The wire layer is deliberately thin. The trust boundary lives entirely inside `admit`, `apply`, and `goalAchieved`.

Question on whether pattern is idiomatic and extensible.

Style.

- Should invariants be `Bool`-valued, or `Prop`-valued with a `Decidable` instance? Which gives cleaner proofs?
- Is the `(InvariantKind × (State → Bool))` registry the right shape, or would a record with a name, predicate, and reason be more idiomatic?

Model.

- `apply` is total, so missing keys become silent no-ops. Should we refactor it to return `Option State`?
- `scaleDeployment` synchronously fabricates pods. Should this become a separate `controllerStep` relation, with `apply` sound with respect to a small-step `Step : State → ApiVerb → State → Prop`?

Future extentions.

- How should invariants compose as we add new verbs? Pointers to separation logic, refinement types, or other approaches.